Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

Definition The Composite pattern is a structural design pattern that allows you to compose objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly

Standard Analogy: File System
Think of a File System. It consists of Files and Folders. A Folder can contain Files or other Folders. Whether you are checking the size of a single File or a Folder containing a hundred items, you perform the same action ("Get Size").

Java

```java
import java.util.ArrayList;
import java.util.List;

// Component Interface
interface FileSystemComponent {
    void showDetails();
}

// Leaf (Individual Object)
class File implements FileSystemComponent {
    private String name;

    public File(String name) { this.name = name; }

    public void showDetails() {
        System.out.println("File: " + name);
    }
}

// Composite (Collection of Objects)
class Folder implements FileSystemComponent {
```

```java
// Composite (Collection of objects)
class Folder implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> components = new ArrayList<>();

    public Folder(String name) { this.name = name; }

    public void addComponent(FileSystemComponent component) {
        components.add(component);
    }

    public void showDetails() {
        System.out.println("Folder: " + name);
        for (FileSystemComponent component : components) {
            component.showDetails(); // Uniform treatment
        }
    }
}

// Test Class
public class CompositePatternTest {
    public static void main(String[] args) {
        // Individual files
        File file1 = new File("resume.pdf");
        File file2 = new File("photo.jpg");

        // Folder containing files
```

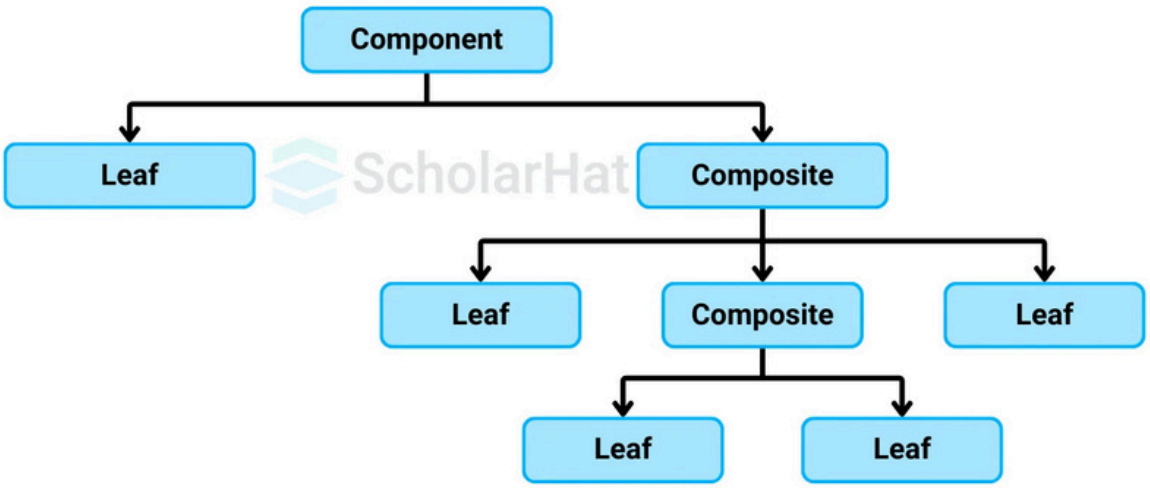
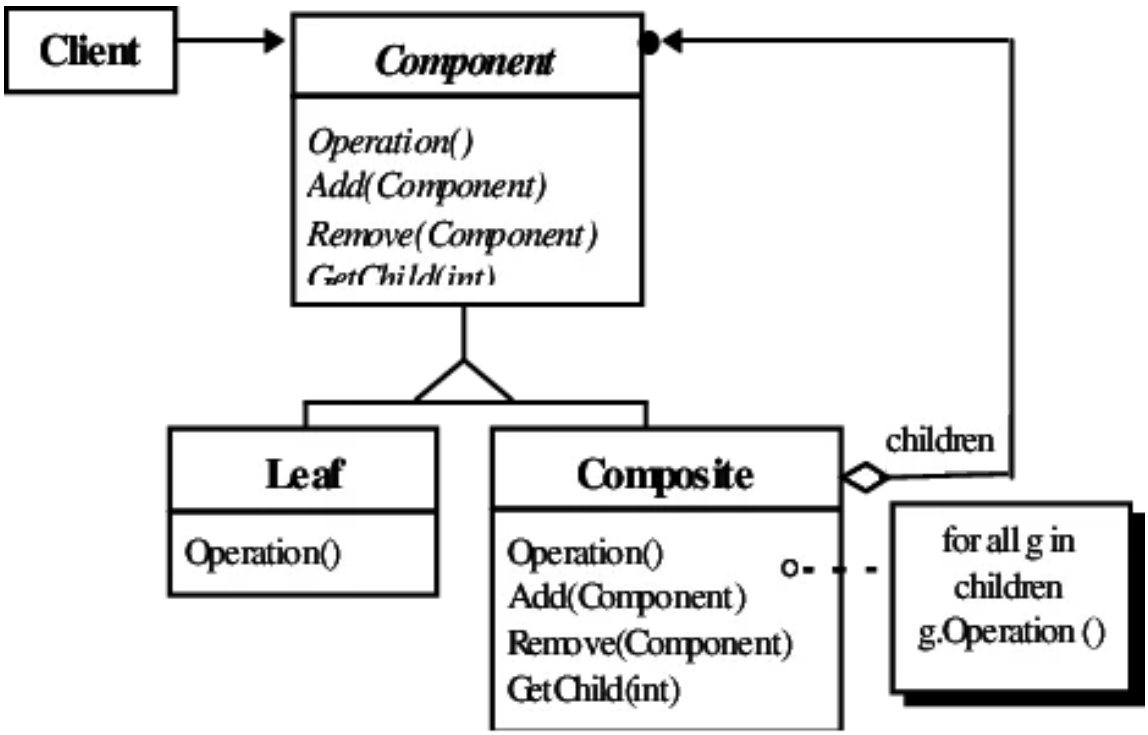
```java
// Test Class
public class CompositePatternTest {
    public static void main(String[] args) {
        // Individual files
        File file1 = new File("resume.pdf");
        File file2 = new File("photo.jpg");

        // Folder containing files
        Folder myDocuments = new Folder("My Documents");
        myDocuments.addComponent(file1);
        myDocuments.addComponent(file2);

        // Nested Folder
        Folder workFolder = new Folder("Work");
        workFolder.addComponent(new File("project.docx"));

        myDocuments.addComponent(workFolder);

        // Client treats Folder and File the same way
        myDocuments.showDetails();
    }
}
```

Why this is a Composite Pattern

- **Recursive Structure:** The `Folder` (Composite) can contain other `FileSystemComponent` objects, which can be either `File` (Leaf) or other `Folder` objects.
- **Uniformity:** The client code in the test class calls `showDetails()` on the `myDocuments` object without needing to check if it is a single file or a collection.
- **Part-Whole Hierarchy:** It perfectly represents a tree structure where "parts" (Files) and "wholes" (Folders) share the same interface.
- **Simplified Client:** The client doesn't have to write complex loops to distinguish between individual items and containers; the Composite handles the iteration internally.